Instituto Federal de Educação, Ciência e Tecnologia de São Paulo
Câmpus Birigui

Henrique José de Souza

Sistema Especialista Educacional para Validação de Compatibilidade de Hardware baseado em Problema de Satisfação de Restrições (CSP).

Birigui
2026

Henrique José de Souza

Sistema Especialista Educacional para Validação de Compatibilidade de Hardware baseado em Problema de Satisfação de Restrições (CSP).

Trabalho de Conclusão de Curso apresentado ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, como requisito parcial para conclusão do curso de Engenharia da Computação.

Área de Concentração: Área de Concentração do Trabalho

Orientador: Profa. Dra. Helen de Freitas Santos

Birigui

2026

FICHA CATALOGRÁFICA (obrigatória; impressa no verso da folha de rosto, não conta na paginação)

Rissi, Viviane Renata Ventura.
A importância da biblioteca : análise dos últimos 7 anos / Viviane R. V. Rissi, 2019
90 f. : il.

Orientador: Nome direto

Monografia (Graduação)- Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Birigui, 2019.

1. Leitura (Ensino superior). 2. Livros (Investimento). 3. Leitura – Meios auxiliares. I. Instituto Federal de Educação, Ciência e Tecnologia. II. Título.

Ficha catalográfica elaborada pela Biblioteca do IFSP – Campus Birigui

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de figura 16

LISTA DE QUADROS

Quadro 1 – Exemplo de quadro 16

LISTA DE TABELAS

Tabela 1 – Exemplo de tabela de 2 colunas 16

SUMÁRIO

1 REVISÃO DA LITERATURA 8
1.1 Compatibilidade de Hardware como Domínio do Problema 8
1.1.1 Compatibilidade de Soquete CPU-Placa-mãe 8
1.1.2 Padrão de Memória RAM 8
1.1.3 Capacidade Energética e TDP 9
1.1.4 Formalização como Problema de Restrições 9
1.2 Problemas de Satisfação de Restrições (CSP) 9
1.2.1 Tipos de Restrições 10
1.2.2 Aplicação ao Domínio de Hardware 10
1.2.3 Representação em Grafo 11
1.3 Teoria dos Grafos e Grafo de Restrições 11
1.3.1 Definições Fundamentais 11
1.3.2 Representações Computacionais 11
1.3.3 Grafo de Restrições 12
1.4 Complexidade Computacional 12
1.4.1 Implicações para o Domínio de Hardware 13
1.5 Algoritmos de Busca e Backtracking 14
1.5.1 Busca em Espaço de Atribuições 14
1.5.2 Backtracking 14
1.5.3 Forward Checking 14
1.6 Propagação de Restrições e Algoritmo AC-3 15

Referências 20

1 REVISÃO DA LITERATURA

1.1 Compatibilidade de Hardware como Domínio do Problema

A montagem de um computador *desktop* exige o cruzamento de especificações técnicas entre componentes interdependentes. Diferentemente da aquisição de periféricos genéricos, a configuração de um sistema computacional envolve restrições físicas, elétricas e lógicas que, quando violadas, resultam em incompatibilidade absoluta: componentes adquiridos de forma incorreta não funcionam juntos, independentemente de ajustes de *software* ou configuração. Esse cenário impõe ao usuário leigo um desafio técnico significativo, uma vez que plataformas comerciais tradicionais não oferecem mecanismos de validação preventiva capazes de explicar o motivo da incompatibilidade (Advanced Micro Devices, 2022; Intel Corporation, 2023).

O presente trabalho concentra-se em cinco categorias de componentes que consitituem o núcleo de qualquer configuração *desktop*: a Unidade Central de Processamento (CPU), a placa-mãe, os módulos de memória RAM, a Unidade de Processamento Gráfico (GPU) e a fonte de alimentação. Cada uma dessas categorias possui especificações técnicas que determinam sua compatibilidade com as demais, formando uma rede de dependências que não pode ser avaliada de forma isolada.

1.1.1 Compatibilidade de Soquete CPU-Placa-mãe

A restrição mais fundamental na montagem de um computador é a compatibilidade de soquete entre o processador e a placa-mãe. Cada geração de processadores é projetada para um soquete físico específico, definido pelo número, disposição e função de seus contatos elétricos. Essa restrição é bínaria e absoluta: um processador projetado para o soquete AM5 da AMD é fisicamente incompatível com uma placa-mãe equipada com soquete AM4, e vice-versa, sem qualquer possibilidade de adaptação ou ajuste (Advanced Micro Devices, 2022). De forma análoga, processadores Intel para o soquete LGA1851 não são compatíveis com plataformas LGA1700 (Intel Corporation, 2023). A documentação técnica oficial dos fabricantes define formalmente quais combinações de processadores e *chipsets* são suportadas, constituindo a principal fonte de restrições binárias do dóminio.

1.1.2 Padrão de Memória RAM

Os módulos de memória RAM seguem padrões técnicos definidos pela JEDEC (*Joint Electron Device Engineering Council*), organização responsável pela padronização internacional de semicondutores. Os padrões DDR4 e DDR5, por exemplo, diferem em

números de pinos, tensão de operação, frequência base e formato físico do enxaixe (JEDEC Solid State Technology Association, 2020). Essa diferença torna os dois padrões eletricamente e fisicamente incompatíveis: uma placa-mãe projetada para DDR5 não aceita módulos DDR4, e o próprio conector possui geometria distinta que impede a inserção física incorreta. A seleção de memória compatível depende, portanto, da placa-mãe selecionada, que por sua vez depende do processador, criando uma cadeia de dependências transitivas no grafo de restrições.

1.1.3 Capacidade Energética e TDP

Cada componente de *hardware* possui um indicador de consumo máximo de energia denominado TDP (*Thermal Design Power*), expresso em watts. A fonte de alimentação deve ser dimensionada para suprir a soma dos TDPs de todos os componentes, com uma margem de seguraça recomendada de aproximadamente 20% a 30% acima do consumo estimado (Intel Corporation, 2022). Uma fonte subdimensionada provoca instabilidade do sistema, reinicializações inesperadas e, em casos extremos, danos permanentes aos componentes. Essa restrição difere das anteriores por ser de natureza quantitativa: não se trata de uma incompatibilidade binária, mas de uma relação de desigualdade entre a capacidade da fonte e a demanda agregada dos componentes.

1.1.4 Formalização como Problema de Restrições

As três categorias de restrições descritas, soquete, padrão de memória e capacidade energética, possuem uma estrutura comum: cada um envolve um par de componentes e define condições que os valores atribuídos a esse par devem satisfazer. Essa estrutura é precisamente o que a teoria dos Problemas de Satisfação de Restrições (CSP) formaliza: cada categoria de componente pode ser modelada como uma variável, o conjunto de modelos comercialmente disponíveis em cada categoria constitui o domínio dessa variáveis. Essa correspondência torna o CSP não apenas uma metáfora convenientem, mas a estrutura formal mais adequada para modelar o problema de validação de configurações de *hardware*, conforme detalhado na seção seguinte.

1.2 Problemas de Satisfação de Restrições (CSP)

Um Problema de Satisfação de Restrições (CSP) é formalmente caracterizado por uma tripla $\langle X, D, C \rangle$, na qual $X = \{X_1, \ldots, X_n\}$ denota um conjunto finito de variáveis, $D = \{D_1, \ldots, D_n\}$ representa os respectivos domínios, de modo que cada D_i delimita os valores admissíveis para X_i, e C reúne as restrições que impõem condições sobre as combinações válidas de valores (Russell; Norvig, 2010). Cada restrição C_j é expressa como um par $\langle \mathit{scope}, \mathit{rel} \rangle$, em que *scope* é uma tupla de variáveis participantes e *rel* define os

valores que essas variáveis podem assumir conjuntamente (Russell; Norvig, 2010). Diz-se que o problema está resolvido quando se obtém uma atribuição completa e consistente: uma função que associa a cada variável um valor de seu domínio sem infringir nenhuma restrição de C.

1.2.1 Tipos de Restrições

As restrições em um CSP podem ser classificadas quanto ao número de variáveis que envolvem. Restrições unárias afetam uma única variável, limitando seu domínio independentemente das demais, por exemplo: a restrição de que a fonte de alimentação deve ter potência mínima de 500W. Restrições binárias envolvem duas variáveis e são as mais comuns em problemas de compatibilidade de *hardware*, por exemplo: a restrição de que o soquete da CPU deve corresponder ao soquete suportado pela placa-mãe. Restrições de ordem superior envolvem três ou mais variáveis simultaneamente (Dechter, 2003). O sistema proposto neste trabalho opera predominantemente com restrições binárias, o que viabiliza a representação estrutural em grafo de restrições.

1.2.2 Aplicação ao Domínio de Hardware

A aplicação dessa formalização ao domínio de compatibilidade de *hardware* é direta. Considerando os cinco componentes centrais do sistema (CPU, placa-mãe, RAM, GPU e fonte), a tripla (X, D, C) pode ser definida da seguinte forma:

- $X = \{\text{CPU}, \text{Placa-mãe}, \text{RAM}, \text{GPU}, \text{Fonte}\}$;
- $D = \{D_{\text{CPU}} = \text{modelos de processadores disponíveis},$
  $D_{\text{Placa}} = \text{modelos de placas-mãe disponíveis},$
  $D_{\text{RAM}} = \text{modelos de memória RAM disponíveis},$
  $D_{\text{GPU}} = \text{modelos de placas de vídeo disponíveis},$
  $D_{\text{Fonte}} = \text{modelos de fontes disponíveis}\}$
- $C = \{\text{soquete}(\text{CPU}, \text{Placa-mãe}),$
  $\text{padrão_memória}(\text{Placa-mãe}, \text{RAM}),$
  $\text{tdp_cpu}(\text{CPU}, \text{Fonte}),$
  $\text{tdp_gpu}(\text{GPU}, \text{Fonte})\}$

Para ilustrar concretamente, considere a atribuição parcial $\{\text{CPU} = \text{Ryzen 5 7600}, \text{Placa-mãe} = \text{B650}, \text{RAM} = \text{DDR5-5600}\}$. Essa atribuição é consistente: o Ryzen 5 7600 utiliza soquete AM5, a placa B650 suporta AM5, e o padrão DDR5 é compatível com a plataforma AM5. Contudo, a atribuição $\{\text{CPU} = \text{Ryzen 5 7600}, \text{Placa-mãe} = \text{B650},$

RAM = DDR4-3200} viola a restrição padrão_memória, pois placas-mãe com chipset B650 suportam exclusivamente DDR5 (Advanced Micro Devices, 2022; JEDEC Solid State Technology Association, 2020). A solução do CSP consiste em encontrar uma atribuição completa, cobrindo todas as cinco variáveis, que satisfaça simultaneamente todas as restrições de C.

1.2.3 Representação em Grafo

A principal distinção entre o CSP e as abordagens clássicas de busca em espaço de estados reside na representação fatorada que aquele adota: em vez de tratar o estado como uma entidade atômica e indivisível, o CSP expõe a estrutura interna do problema por meio de variáveis e restrições explícitas (Russell; Norvig, 2010; Kumar, 1992). Restrições binárias entre pares de variáveis admitem representação natural como grafo, no qual os vértices correspondem às variáveis e as arestas codificam as restrições. Essa estrutura de grafo de restrições é o objeto central sobre o qual os algoritmos de busca e propagação operam.

1.3 Teoria dos Grafos e Grafo de Restrições

1.3.1 Definições Fundamentais

Um grafo é formalmente definido como um par ordenado $G = (V, E)$, onde V é um conjunto finito não vazio de vértices e $E \subseteq V \times V$ é um conjunto de arestas (Diestel, 2017). Um grafo é dito simples quando não contém laços (arestas de um vértice para si mesmo) nem arestas paralelas (múltiplas arestas entre o mesmo par de vértices). Em um grafo não direcionado, as arestas não possuem orientação: a aresta $\{u, v\}$ é idêntica a $\{v, u\}$. Dois vértices u e v são ditos adjacentes se existe uma aresta $\{u, v\} \in E$ conectando-os (Diestel, 2017).

1.3.2 Representações Computacionais

Do ponto de vista computacional, um grafo pode ser representado de duas formas principais (Cormen *et al.*, 2001). A matriz de adjacência é uma matriz A de dimensão $|V| \times |V|$, na qual $A[i][j] = 1$ se existe aresta entre os vértices i e j, e 0 caso contrário. Essa representação permite verificar a existência de uma aresta em tempo constante $O(1)$, mas exige espaço $O(|V|^2)$, independentemente do número de arestas. A lista de adjacência associa a cada vértice uma lista de seus vizinhos, exigindo espaço $O(|V| + |E|)$. Para grafos esparsos, onde o número de arestas é significativamente menor que V^2, a lista de adjacência é preferível em termos de eficiência de memória (Cormen *et al.*, 2001).

O grafo de restrições do sistema proposto é esparso por natureza: com cinco vértices (componentes), o número máximo de arestas seria 10, mas o sistema possui apenas quatro restrições binárias efetivas, soquete entre CPU e Placa-mãe, padrão de memória entre Placa-mãe e RAM, TDP da CPU e Fonte, e TDP da GPU e Fonte. Pares como GPU e RAM ou CPU e GPU, não possuem restrições física direta e portanto não são conectados por arestas. A lista de adjacência é, portanto, a representação mais adequada para esse grafo.

1.3.3 Grafo de Restrições

Dado um CSP com variáveis X e restrições C, o grafo de restrições $G_C = (X, E)$ é construído de modo que existe uma aresta $\{X_i, X_j\} \in E$ se e somente se existe alguma restrição C envolvendo simultaneamente as variáveis X_i e X_j (Russell; Norvig, 2010). Essa representação explicita a estrutura topológica do problema, vertíces isolados correspondem a variáveis sem restrições com outras, e componentes fortemente conectados indicam grupos de variáveis com alta interdependência.

Aplicando essa definição ao sistema proposto, o grafo de restrições possui a seguinte estrutura:

- **Vértices:** $V = \{\text{CPU}, \text{Placa-mãe}, \text{RAM}, \text{GPU}, \text{Fonte}\}$
- **Arestas:** E compreende as seguintes relações:
  - $\{\text{CPU}, \text{Placa-mãe}\}$, restrição de soquete;
  - $\{\text{Placa-mãe}, \text{RAM}\}$, restrição de padrão de memória;
  - $\{\text{CPU}, \text{Fonte}\}$, restrição de TDP do processador;
  - $\{\text{GPU}, \text{Fonte}\}$, restrição de TDP da placa de vídeo.

Essa estrutura revela que a Fonte é um vértice de alta centralidade no grafo, pois está conectada tanto à CPU quanto à GPU, de modo que qualquer alteração no domínio de um desses componentes pode propagar restrições para o domínio da Fonte. A Placa-mãe, por sua vez, é o vértice que medeia a compatibilidade entre CPU e RAM, funcionando como ponto de articulação da rede de restrições. Esse grafo é o objeto sobre o qual os algoritmos de busca e propagação operam, cuja complexidade computacional é analisada na seção seguinte.

1.4 Complexidade Computacional

A teoria da complexidade computacional ocupa-se da classificação de problemas de decisão segundo os recursos algorítmicos, principalmente tempo de execução e espaço de

memória, exigidos para sua resolução (Cormen *et al.*, 2001). Dentro desse arcabouço, a classe P reúne os problemas para os quais existem algoritmos determinísticos de tempo polinomial, considerados eficientes e tratáveis mesmo para entradas de grande escala. A classe NP, por sua vez, abrange os problemas cuas soluções propostas podem ser verificados em tempo polinomial por uma máquina determinística, ainda que não se conheça, até o presente, algoritmo eficiente para encontrá-las (Sipser, 2012). A questão sobre a relação entre essas duas classes, se $P = NP$, permanece em aberto e constitui o problema não resolvido de maior relavância para a ciência da computação teórica, razão pela qual o *Clay Mathematics Institute* o incluiu entre os sete Problemas do Milênio (Sipser, 2012).

O interior da classe NP não é uniforme. A literatura demonstra a existência de subclasses com diferentes graus de dificuldade, incluindo problemas resolúveis em tempo subexponencial por meio de não-determinismo limitado (Papadimitriou; Yannakakis, 1996). O CSP, contudo, não se enquadra nessas classes intermediárias, situando-se no topo da hierarquia como um problema NP-completo (Sipser, 2012). Um problema é classificado como NP-completo quando pertence a NP e todo problema NP se reduz a ele em tempo polinomial, essa classificação pode ser estabelecida por redução polinomial a partir do Problema da Satisfabilidade Booleana (SAT), um dos problemas canônicos de NP-completude (Cormen *et al.*, 2001; Sipser, 2012). Consequentemente, se qualquer problema NP-completo admitisse solução em tempo polinomial, toda classe NP colapsaria em P (Sipser, 2012).

1.4.1 Implicações para o Domínio de Hardware

Em natureza NP-completa impõe restrições severas à arquitetura de qualquer sistema de verificação de compatibilidade de *hardware* fundamentado em CSP. A primeira limitação é de ordem temporal, a avaliação exaustiva de todas as combinações interdependentes de componentes é inviável na prática em razão da explosão combinatória intrínseca à classe (Borodin; Cashman; Magen, 2011). Para ilustrar concretamente, considere que cada categoria de componente possui em média 50 modelos disponíveis no catálogo do sistema. Com cinco categorias (CPU, Placa-mãe, RAM, GPU e Fonte), a avaliação por força bruta implicaria examinar $50^5 = 312.500.000$ atribuições candidatas antes de encontrar uma solução válida no pior caso. Esse número cesce exponencialmente à medida que o catálogo se expande, tornando a abordagem inviável em tempo hábil.

A segunda limitação é de ordem espacial, estratégias alternativas baseadas no mapeamento estático e exaustivo de regras lógicas condicionais, codificando explicitamente cada restrição de *hardware* por meio de estruturas if-else ou tabela de decisão, não contornam o problema, pois a representação de funções lógicas complexas por árvores de decisão pode exiger espaço de tamanho exponencial (Jukna *et al.*, 1999). Diante dessas limitações, a abordagem padrão para tornar CSPs tratáveis na prática é a busca sistemática

com retrocesso combinada com técnicas de propagração de restrições, que reduzem o espaço efetivo de busca de forma incremental.

1.5 Algoritmos de Busca e Backtracking

1.5.1 Busca em Espaço de Atribuições

Dado o grafo de restrições e a complexidade NP-completa do CSP, é necessário um algoritmo de busca sistemática que explore o espaço de atribuições de forma inteligente, sem enumerar cegamente todas as combinações possíveis. O ponto de partida é o conceito de atribuição parcial, uma função que associa valores a um subconjunto das variáveis, podendo ser progressivamente estendida até uma atribuição completa (Russell; Norvig, 2010). A ideia central dos algoritmos de busca para CSP é construir a atribuição incrimentalmente, verificando consistência a cada passo e abandonando ramos da árvore de busca assim que uma inconsistência é detectada.

1.5.2 Backtracking

O algoritmo de *backtracking* é o método fundamental de busca para CSPs (Russell; Norvig, 2010; Kumar, 1992). Ele percorre recursivamente a árvore de atribuições, em cada nível da recursão, seleciona uma variável ainda não atribuida, tenta cada valor do seu domínio na ordem disponível, verifica se esse valor é consistente com as restrições envolvendo as variáveis já atribuidas e recua (*backtrack*) ao encontrar uma inconsistência. O pseudocódigo do algorimto é apresentado a seguir:

A complexidade de pior caso do *backtracking* puro é $O(d^n)$, na qual d é o tamanho máximo dos domínios e n é o número de variáveis, reiterando a inviabilidade para instâncias grandas sem otimizações complementares (Russell; Norvig, 2010).

1.5.3 Forward Checking

Uma primeira melhoria ao backtracking puro é o *forward checking* (Dechter, 2003). Ao atribuir um valor à variável X_i, o algoritmo imediatamente examina cada variável X_j, ainda não atribuida que compartilha uma restrição com X_i e remove de $Dom(X_j)$ todos os valores que se tornam inconsistentes com a atribuição de X_i. Se algum domínio X_j se torna vazio, o algoritmo detecta a falha antecipadamente e recua, sem precisar explorar o ramo até o fim.

Aplicando ao domínio de *hardware*, ao selecionar uma CPU com soquete AM5, o *forward checking* percorre os vizinhos de CPU no grafo de restrições e remove do domínio de Placa-mãe todos os modelos com soquete incompatível com AM5, antes mesmo de tentar atribuir um valor a Placa-mãe. Essa poda antecipada reduz significativamente o

Algoritmo 1 – Algoritmo de Busca por Backtracking para CSP

```
1: função BACKTRACKING-SEARCH(csp)
2:     retornar BACKTRACK(∅, csp)
3: fim função

4: função BACKTRACK(atribuicao, csp)
5:     se atribuicao está completa então
6:         retornar atribuicao
7:     fim se
8:     var ← SELECIONAR-VARIAVEL-NAO-ATRIBUIDA(csp)
9:     para cada valor em ORDENAR-VALORES(var, atribuicao, csp) faça
10:        se valor é consistente com atribuicao dado csp então
11:            adicionar {var = valor} à atribuicao
12:            resultado ← BACKTRACK(atribuicao, csp)
13:            se resultado ≠ falha então
14:                retornar resultado
15:            fim se
16:            remover {var = valor} da atribuicao
17:        fim se
18:    fim para
19:    retornar falha
20: fim função
```

Fonte: O autor (2026).

número de atribuições exploradas. Contudo, o *forward checking* verifica apenas os vizinhos diretos da variável recém-atribuida, sem propagar as consequências dessas remoções para variáveis mais distantes no grafo. A forma mais completa e eficiente de propagação, que garante consistência global na rede, é o Algoritmo AC-3.

1.6 Propagação de Restrições e Algoritmo AC-3

A principal técnica para contornar a explosão combinatória é a propagação de restrições, realizada por algoritmos de consistência. O mais consolidado na literatura é o AC-3 (*Arc Consistency Algorithm 3*), proposto por (Mackworth, 1977), cujo objetivo é garantir a consistência de arco em redes de restrições binárias (Kumar, 1992). Diz-se que um arco (X_i, X_j) é consistente quando, para todo valor $v \in Dom(X_i)$, existe ao menos um valor $w \in Dom(X_j)$ tal que a atribuição $X_i = v, X_j = w$ satifaz a restrição entre as duas variáveis.

O algoritmo mantém uma fila de arcos a examinar e, para cada par de variáveis relacionadas, remove do domínio da variável os valores que não encontram suporte em nenhum valor do domínio da vaariável vizinha. Cada remoção dispara a reinserção dos

arcos afetados na fila, propagando a mudança pela rede até o ponto de estabilidade, em que nenhuma redução adicional é possível (Russell; Norvig, 2010). Assumindo um CSP com n variáveis, domínios de tamanho máximo d e c restrições binárias, a complexidade de píor caso do AC-3 é $O(cd^3)$ (Russell; Norvig, 2010).

Texto da revisão da literatura, dividido em seções e subseções.

Este é um exemplo de como usar figuras. Referência cruzada: Figura 1

Figura 1 – Exemplo de figura

Fonte: Elaborada pelo autor

Este é um exemplo de como usar tabelas. Referência cruzada: Tabela 1

Tabela 1 – Exemplo de tabela de 2 colunas

Coluna 1	**Coluna 2**
Dado 1a	Dado 2a
Dado 1b	Dado 2b
Dado 1c	Dado 2c
Dado 1d	Dado 2d

Fonte: Elaborada pelo autor

Este é um exemplo de como usar quadros. Referência cruzada: Quadro 1

Quadro 1 – Exemplo de quadro

Cores		
Nome	**Hexa**	**Amostra**
Preto	#000000	
Marrom	#993300	
Vermelho	#FF0000	
Laranja	#FF3300	
Amarelo	#FFFF00	
Branco	#FFFFFF	

Fonte: Elaborada pelo autor

Este é um exemplo de como usar equações. Referência cruzada: Equação 1.1

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \tag{1.1}$$

Exemplo de inserção de lista de código fonte (**não use acentos no código!**):

```
/**
 * Classe de exemplo.
 *
 * @author David Buzatto
 */
public class Grafo {

    public static void main( String[] args ) {

        System.out.println( "Exemplo de codigo fonte!" );
        System.out.println( "Nao use acentos!" );

    }

}
```

Este é um exemplo de como inserir texto sem formatação (ambiente verbatim):

```
Texto sem formatação, como espaçamento igual.
```

Exemplo de lista de itens:

- **Item 1:** texto...;
- **Item 2:** texto...;
  - **Subitem:** texto...;
  - **Subitem:** texto...;
  - **Subitem:** texto...;
- **Item 3:** texto...;
- **Item n:** texto....

Exemplo de lista numerada:

1. **Item:** texto...;
2. **Item:** texto...;
   a) **Subitem:** texto...;
   b) **Subitem:** texto...;
   c) **Subitem:** texto...;

3. **Item:** texto...;
4. **Item:** texto....

Exemplos de comandos para texto e referências:

- Para iniciar um novo parágrafo, basta deixar uma linha em branco no código fonte;
- Não force o compilador a pular mais de uma linha, pois terá influência negativa na composição do documento;
- Sempre deixe o LaTeX realizar a formatação de parágrafos e posicionamento de elementos;
- Utilização de aspas simples (abertura `‘`, fechamento `’`): ‘Texto entre aspas simples’;
- Utilização de aspas duplas (abertura `‘‘`, fechamento `’’`): “Texto entre aspas duplas”;
- Negrito (comando `\textbf`): **texto em negrito**;
- Itálico (comando `\textit`): *texto em itálico*;
- Sublinhado (comando `\underline`): texto sublinhado;
- Negrito e itálico (usar comandos juntos): ***texto em negrito e itálico***;
- Alterar cor do texto (comando `\textcolor{cor}{texto}`):
  - Exemplo `\textcolor{red}{texto}`: texto vermelho;
  - Exemplo `\textcolor[RGB]{255, 102, 0}`: texto laranja;
  - Exemplo `\textcolor[HTML]{006AD7}`: texto azul;
- Ambiente matemático inline (comando `$ expressão $`): $s = x^2 - 2x + 1$;
- Referência normal (comando `\cite`):
  - (**Agaisse1995**);
  - (**Abedi2014**);
  - (**BtNomenclature2016**);
- Referência normal com mais de uma obra (comando `\cite`):
  - (**Abedi2014**; **Agaisse1995**);
  - (**AgapitoTenfen2014**; **BtNomenclature2016**; **Nelson2014**);
- Referência nome e ano (comando `\citeauthorandyear`):

- **Agaisse1995**;
- **Abedi2014**;
- **BtNomenclature2016**;

Exemplo 1 de citação direta:

> Os 20 aminoácidos usualmente encontrados como resíduos em proteínas contém um grupo α-carboxil, um grupo α-amino e um grupo R distinto substituído no átomo de carbono α. O átomo de carbono α de todos os aminoácidos, com exceção da glicina, é assimétrico e, portanto, os aminoácidos podem existir em pelo menos duas formas estereoisoméricas. Somente os estereoisômeros L, com uma configuração relacionada à configuração absoluta da molécula de referência L-gliceraldeído, são encontrados em proteínas (**Nelson2014**).

Exemplo 2 de citação direta:

> *These various insecticidal proteins are synthesized during the stationary phase and accumulate in the mother cell as a crystal inclusion which can account for up to 25% of the dry weight of the sporulated cells. The amount of crystal protein produced by a B. thuringiensis culture in laboratory conditions (about 0.5 mg of protein per ml) and the size of the crystals (24) indicate that each cell has to synthesize* 10^6 *to* 2×10^6 *δ-endotoxin molecules during the stationary phase to form a crystal* (**Agaisse1995**).

Exemplo de nota de rodapé[1].

[1] Essa é uma nota de rodapé!

REFERÊNCIAS

ADVANCED MICRO DEVICES. **Meet the New AMD Socket AM5 Platform**. [*S. l.: s. n.*], 2022. AMD Partner Insights. Acesso em: abr. 2026. Disponível em: `https://www.amd.com/en/partner/articles/socket-am5-change-game.html`. Citado nas pp. 8, 11.

BORODIN, Allan; CASHMAN, David; MAGEN, Avner. How Well Can Primal-Dual and Local-Ratio Algorithms Perform? **ACM Transactions on Algorithms**, v. 7, n. 3, 2011. ISSN 1549-6325. DOI: `10.1145/1978782.1978784`. Citado na p. 13.

CORMEN, Thomas H. *et al.* **Introduction to Algorithms**. 2. ed. Cambridge, MA: The MIT Press, 2001. ISBN 9780262032937. Citado nas pp. 11, 13.

DECHTER, Rina. **Constraint Processing**. San Francisco, CA: Morgan Kaufmann, 2003. ISBN 9781558608900. Citado nas pp. 10, 14.

DIESTEL, Reinhard. **Graph Theory**. 5. ed. Berlin: Springer, 2017. ISBN 9783662575604. Disponível em: `https://diestel-graph-theory.com`. Citado na p. 11.

INTEL CORPORATION. **ATX Specification Version 3.0**. [*S. l.*], 2022. Disponível em: `https://edc.intel.com`. Citado na p. 9.

INTEL CORPORATION. **LGA1851 and LGA1700 Socket Incompatibility**. [*S. l.: s. n.*], 2023. Intel Support Knowledge Base, Article ID: 000099723. Acesso em: abr. 2026. Disponível em: `https://www.intel.com/content/www/us/en/support/articles/000099723/processors.html`. Citado na p. 8.

JEDEC SOLID STATE TECHNOLOGY ASSOCIATION. **JESD79-5B: DDR5 SDRAM Standard**. Arlington, VA, 2020. Disponível em: `https://www.jedec.org`. Citado nas pp. 9, 11.

JUKNA, Stasys *et al.* On P versus NP ∩ co-NP for Decision Trees and Read-Once Branching Programs. **Computational Complexity**, v. 8, n. 4, p. 357–370, 1999. DOI: `10.1007/s000370050005`. Citado na p. 13.

KUMAR, Vipin. Algorithms for Constraint-Satisfaction Problems: A Survey. **AI Magazine**, v. 13, n. 1, p. 32–44, 1992. DOI: `10.1609/aimag.v13i1.976`. Disponível em: `https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/976`. Citado nas pp. 11, 14, 15.

MACKWORTH, Alan K. Consistency in Networks of Relations. **Artificial Intelligence**, v. 8, n. 1, p. 99–118, 1977. ISSN 0004-3702. DOI: 10.1016/0004-3702(77)90007-8. Citado na p. 15.

PAPADIMITRIOU, Christos H.; YANNAKAKIS, Mihalis. On Limited Nondeterminism and the Complexity of the V-C Dimension. **Journal of Computer and System Sciences**, v. 53, n. 2, p. 161–170, 1996. ISSN 0022-0000. DOI: 10.1006/jcss.1996.0058. Disponível em: https://www.sciencedirect.com/science/article/pii/S0022000096900586. Citado na p. 13.

RUSSELL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 3. ed. Upper Saddle River, NJ: Prentice Hall, 2010. ISBN 9780136042594. Citado nas pp. 9–12, 14, 16.

SIPSER, Michael. **Introduction to the Theory of Computation**. 3. ed. Stamford, CT: Cengage Learning, 2012. ISBN 9781133187790. Citado na p. 13.